

Práctica 7: Diagonalización Por Método De Gauss, Factorización LU y Factorización QR

Ejercicio 1:

En este ejercicio deberán implementar el método de eliminación Gaussiana para diagonalizar una matriz cuadrada.

Paso 1:

Programa una función que reciba una Matriz cuadrada de tamaño $n \times n$, M , y que por medio del método de eliminación Gaussiana regrese una **Matriz triangular superior**. Una vez que hayas terminado prueba tu función con distintas matrices cuadradas usando el comando.

```
M = np.random.random((n,n))````  
con diferentes valores para $n$
```

```
![Matriz triangular superior ]  
(https://sites.google.com/site/trabajoprocesos20151/\_/rsrc/1429135824482/ecuaciones-de-una-variable/1-metodos/1-1-eliminacion-gaussiana-simple/1.PNG)
```

Pseudocodigo:

Para cada renglon i:

Se utiliza el renglon i de la matriz como fila pivote o fila de referencia, R_i

para cada renglon j:

donde j varía desde $i+1$, ..., n.

Dividimos al pivote entre el valor a_{ii} para así tener un 1 en la entrada i y lo

multiplicamos por una cantidad $-a_{ij}$.

Finalmente, reemplazamos $R_j = R_j + cR_i = R_j + cR_i$ donde $c = -a_{ij}/a_{ii}$

...

Cuidado!! Hay que tener cuidado cuando hay un 0 en la diagonal. Lo más fácil es suponer que eso no va a pasar y programar el método de eliminación Gaussiana omitiendo ese hecho; pero lo correcto es corroborar que eso no pase llevando a cabo un intercambio con alguno de los renglones debajo del renglón que tiene 0 en la diagonal. Intenta resolver el

primer caso y luego agregarle el paso en el que te aseguras que no haya ceros en la diagonal.

Problema a considerar. $A = \begin{bmatrix} 1 & 2 & -2 & 1 \\ 4 & 5 & -7 & 6 \\ 5 & 25 & -15 & -3 \\ 6 & -12 & -6 & 22 \end{bmatrix}$

$$A \sim \begin{bmatrix} 1 & 2 & -2 & 1 \\ 0 & -3 & 1 & 2 \\ 0 & 15 & -5 & -8 \\ 0 & -24 & 6 & 16 \end{bmatrix} \sim \begin{bmatrix} 1 & 2 & -2 & 1 \\ 0 & -3 & 1 & 2 \\ 0 & 0 & 0 & 2 \\ 0 & 0 & -2 & 0 \end{bmatrix}$$

Método de Gauss (Pivote Parcial)

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ 0 & a_{22}^1 & a_{23}^1 & a_{24}^1 \\ 0 & a_{32}^1 & a_{33}^1 & a_{34}^1 \\ 0 & a_{42}^1 & a_{43}^1 & a_{44}^1 \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{pmatrix}$$

Pivote Máximo

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ 0 & a_{42}^1 & a_{43}^1 & a_{44}^1 \\ 0 & a_{32}^1 & a_{33}^1 & a_{34}^1 \\ 0 & a_{22}^1 & a_{23}^1 & a_{24}^1 \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} b_1 \\ b_4 \\ b_3 \\ b_2 \end{pmatrix}$$

1. Se busca en la columna j de la matriz el mayor número **en valor absoluto** y dicho valor se ubica.
2. Se pasa intercambian renglones de la matriz y se procede a hacer la eliminación Gaussiana.

```
import numpy as np
```

```
def EliminacionGaussiana(M):
```

```
    # Aqui va su codigo
```

Paso 2:

Programa una función que reciba una Matriz cuadrada de tamaño $n \times n$, M , que además sea triangular superior (como la que obtenemos al usar la función que programaste en el paso 1) y que se encargue de eliminar todos los elementos superiores a la diagonal usando una vez más el método de eliminación Gaussiana.

Paso 3:

Finalmente, programa una función que reciba una Matriz cuadrada de tamaño $n \times n$, M , y que mande a llamar las dos funciones que programaste anteriormente, de forma que al final regrese la diagonal de la matriz que obtuviste al final del procedimiento

def Diag(D) :

Ejercicio 2:

En este ejercicio deberán implementar el método para encontrar la factorización LU de una matriz cuadrada.

Paso 1:

Programen una función que calcule la factorización LU de una matriz cuadrada, A , de tamaño $n \times n$ y que regrese tanto la matriz triangular inferior L , como la matriz triangular superior U . Una vez que hayas terminado prueba tu función con distintas matrices cuadradas usando el comando python `A = np.random.random((n,n))` con diferentes valores para n

1. La matriz L inicialmente es la matriz identidad con el mismo número de renglones de A . Si se utilizó la operación $R_i = R_i + c \cdot \text{Pivote}_j$ entonces en la posición $L[i, j]$ se coloca $-c$.
2. La matriz U es la matriz que queda en al escalar A .

Información: Cuando no es posible factorizar una matriz de la forma LU , lo que se aplica es la factorización $P \cdot A = L \cdot U$, donde P es una matriz de permutación.

Paso 2:

Corroborar que el producto de matrices LU regresa la matriz original A a la que se le encontró la factorización.

Para llevar a cabo el producto de matrices LU puedes utilizar la siguiente línea de código `L.dot(U)`

```
print(A)
print('*****')
print(np.dot(L, U))
```

Ejercicio 3:

En este ejercicio te toca programar la descomposición QR de una matriz A

La idea es usar el **Proceso de Gram-Schmidt**:

A partir de los vectores linealmente independientes $v_1, \dots, v_n$ se construye $u_1, \dots, u_n$ vectores normales.

$$u_1 = v_1, \quad u_k = v_k - \sum_{j=1}^{k-1} \frac{\langle v_k, u_j \rangle}{\|u_j\|^2} u_j, \quad k = 2, \dots, n$$

$$e_i = \frac{u_i}{\|u_i\|}$$

Recordatorio: $\|u\|^2 = \langle u, u \rangle$

Paso 1:

Programa una función que calcule la proyección de un vector u sobre otro vector v

Paso 2:

Programa una función que calcule la descomposición QR de una matriz A y que regrese las matrices Q y R .

$$A = [a_1 \mid a_2 \mid \cdots \mid a_n] = [e_1 \mid e_2 \mid \cdots \mid e_n] \begin{bmatrix} a_1 \cdot e_1 & a_2 \cdot e_1 & \cdots & a_n \cdot e_1 \\ 0 & a_2 \cdot e_2 & \cdots & a_n \cdot e_2 \\ 0 & 0 & \ddots & \vdots \\ 0 & 0 & \cdots & a_n \cdot e_n \end{bmatrix}$$

Paso 3:

Prueba tu función con diferentes matrices y corrobora que el producto de matrices QR regresa la matriz original A a la que se le encontró la factorización.

Para llevar a cabo el producto de matrices QR puedes utilizar la siguiente línea de código `Q.dot(r)`